

CM Track 23: Benachrichtigungswege — Push, Postfach, Warteliste, Einwilligung

Datum: 2026-08-28 **Stack:** Next.js 15, React 19, TypeScript 5.9, Supabase SDK 2.98, Stripe 20.4
Ausgangsstand: bf6fcf6 (Track 22) bzw. 5af4013 (Ledger-Nachtrag) **Tests:** 1526 → 1574 (48 neue, 2 angepasste), alle grün **Gegenprobe:** mit auf bf6fcf6 zurückgesetztem Produktivcode fallen **21 der 48** neuen Tests durch

Zusammenfassung

Geprüft wurden die vier Wege, auf denen die Plattform einen Menschen **erreicht** oder seinen **Willen festhält**:

Weg	Route(n)	Tabelle
Push-Benachrichtigung	/api/push/subscribe , /api/push/send	push_subscriptions
In-App-Postfach	/api/notifications	notification_log
Warteliste	/api/wait-list	wait_list
Cookie-Einwilligung	/api/cookies/consent	cookie_consent

Diese Domäne war in den Tracks 1–22 nie zusammenhängend angesehen worden. Sie hat eine Eigenschaft, die sie von den Geldstrecken unterscheidet und die jeden Fehler hier besonders lange am Leben hält: **niemand vermisst eine Benachrichtigung, die nie ankommt**. Eine ausgefallene Zahlung fällt am selben Tag auf. Ein Wartelisten-Eintrag, der nirgends landet, fällt nie auf — am wenigsten dem, der sich eingetragen hat, denn er hat eine Bestätigung gesehen.

Sieben Befunde: sechs im Code behoben, einer zusätzlich als committete Migration abgesichert. Drei davon haben dieselbe Ursache, und zwar wörtlich dieselbe:

42P10 — *there is no unique or exclusion constraint matching the ON CONFLICT specification*

Postgres verlangt für `INSERT ... ON CONFLICT (a, b)` einen UNIQUE-Index auf **genau** diesen Spalten. Findet es keinen, ist das kein stiller Rückfall auf ein gewöhnliches `INSERT`, sondern ein harter Fehler. An drei Stellen im Produktivcode stand ein `upsert(..., { onConflict: ... })`, dessen Zielspalten zu keinem tauglichen Index passten — jedes Mal bei **jedem** Aufruf, seit Monaten, ohne dass irgendwo etwas rot geworden wäre.

Warum es nie aufgefallen ist, liegt am Werkzeug und ist der eigentliche Nachtrag dieses Tracks: **beide Test-Fakes im Repo konnten ON CONFLICT gar nicht falsch werden lassen**. `src/test/fake-supabase.ts` kannte überhaupt kein `upsert()` — wer einen Test dafür schreiben wollte, bekam `upsert is not a function` und ließ es bleiben. Die zweite Fassung im e2e-Harness nahm die Konfliktspalten einfach als Schlüssel, ohne zu prüfen, ob es dazu einen Index gibt. Beide sind in diesem Track nachgezogen.

Die Befunde

CM23-01 (P1) — Die Warteliste ist ein schwarzes Loch mit Erfolgsmeldung

`POST /api/wait-list` ist öffentlich (Middleware-Whitelist) und die einzige Stelle, an der die Plattform Interessenten in Städten ohne Angebot einsammelt. Sie schrieb:

```
.upsert({ email, city, source, ip, created_at }, { onConflict: 'email,city' })
```

Der einzige UNIQUE-Index der Tabelle ist aber ein **Ausdrucks-Index**:

```
CREATE UNIQUE INDEX wait_list_email_city_uidx  
ON public.wait_list (email, COALESCE(city, ''));
```

— `supabase/migrations/20260515_wait_list.sql`. Ein Ausdrucks-Index ist kein Kandidat für `ON CONFLICT (email, city)`. **Jeder** Eintrag lief in 42P10.

Der Rückfall darunter hat den Ausfall nicht aufgefangen, sondern verdeckt, und zwar aus zwei unabhängigen Gründen:

- Er schrieb nach `newsletter`. Das ist live eine **VIEW** (Sonde: „*permission denied for view newsletter*“), kein Tisch — ein `ON CONFLICT` ist darauf gar nicht möglich.
- Er stand in einem `try/catch`. **supabase-js wirft bei einem Datenbankfehler nicht**, es gibt `{ error }` zurück. Der Rückgabewert wurde nicht angesehen, das `catch` also nie betreten. (Dieselbe Bauart wie CM20-05 beim Bewertungs-Cron.)

Danach lief die Funktion weiter bis `return NextResponse.json({ ok: true })`.

Folge: Seit dem 15.05.2026 hat jeder, der sich eingetragen hat, eine Bestätigung gesehen und steht nirgends. Die Tabelle `wait_list` existiert live und ist leer geblieben; `notified_at` ist eine Spalte, die nie jemand füllen konnte.

Behoben: nachsehen, dann schreiben — ohne `ON CONFLICT`, damit es gegen das Schema läuft, das heute wirklich da ist. Ein `23505` aus dem Rennen zweier gleichzeitiger Anmeldungen zählt als Erfolg (der Endzustand stimmt), jeder andere Fehler wird als **503** gemeldet statt als `ok`. Der tote `newsletter`-Rückfall ist entfernt. Zusätzlich: die Stadt wird auf **eine** Schreibweise normalisiert (leer → `NULL`), und der Zähler des IP-Limits winkt bei einem Ausfall nicht mehr durch.

Nicht live nachweisbar, und das gehört gesagt: Der Beleg ist statisch — Migrationstext plus Postgres-Semantik. Die Tabelle steht live mit exakt den Spalten dieser Migration, sie ist also gelaufen; den Index selbst kann diese Session ohne DB-Zugang nicht auslesen. Ein Live-Test wäre nur mit Schreiben in die Produktionsdatenbank zu haben und wurde deshalb unterlassen.

CM23-02 (P2) — Die Push-Anmeldung schlägt zu 100 % fehl, aus zwei Gründen

`saveSubscription` schrieb:

```
.upsert({ user_id, endpoint, p256dh, auth, updated_at }, { onConflict: 'user_id,endpoint' })
```

1. `push_subscriptions.updated_at` gibt es live nicht. Spaltensonde vom 28.08.2026: ?
`select=updated_at` → 42703, während `created_at` mit 42501 antwortet (existiert, keine Leserechte). Die anlegende Migration `20260317_payments_and_compliance.sql` führt die Spalte ebenfalls nicht. 2. **Der einzige UNIQUE-Index steht auf `endpoint` allein** (`endpoint TEXT NOT NULL UNIQUE`). `ON CONFLICT (user_id, endpoint)` findet dafür keinen Arbitr → 42P10.

Die Funktion warf, die Route antwortete **500**. `push_subscriptions` konnte nie eine Zeile bekommen. Damit lieferte `sendPushNotification` ausnahmslos `{ sent: 0, failed: 0 }` — und `/api/push/send` meldete dem Admin `{ success: true, sent: 0 }`.

Ein `upsert` hätte hier übrigens auch nach der Reparatur die falsche Antwort gegeben: bei einem Endpunkt, der bereits einem **anderen** Konto gehört, hätte er die Zeile stillschweigend umgehängt. Die Benachrichtigungen des Angreifers wären danach auf dem Gerät des Opfers gelandet, dessen eigene nirgends mehr.

Behoben: nachsehen, dann schreiben; keine `updated_at`; ein fremder Endpunkt wird mit **409** abgelehnt statt übernommen; ein Deckel von 20 Geräten je Konto; ein eigenes Rate-Limit (10/h je Konto). Die Migration trägt `updated_at` nach — nicht, weil der Code sie wieder schreiben soll, sondern weil „wann zuletzt gesehen“ die Grundlage jedes späteren Aufräumens abgelaufener Abos ist.

CM23-03 (P2) — Der Push-Endpunkt war eine frei wählbare URL, die der Server abruft

`POST /api/push/subscribe` nahm als `endpoint` **jede** Zeichenkette bis 2000 Zeichen an.
`sendPushNotification` macht daraus:

```
await fetch(sub.endpoint, { method: 'POST', headers: { ...vapidHeaders }, ... })
```

Ein angemeldetes Konto bestimmte damit, wohin unser Server aus dem Rechenzentrum heraus eine Anfrage schickt: `http://169.254.169.254/...` (Metadaten des Hosters),
`http://127.0.0.1:3000/api/...` (unsere eigenen Routen, von innen gesehen), jede interne Adresse. Die Antwort sieht der Angreifer nicht — blind, aber eine Anfrage aus unserem Netz ist die halbe Miete, und der VAPID-Header (ein auf unseren Namen signiertes Token) geht mit.

Erreichbar war das bis heute **nur deshalb nicht**, weil CM23-02 die Tabelle leer hielt. Genau deshalb gehören beide in denselben Track: wer CM23-02 allein repariert, schaltet CM23-03 scharf.

Behoben: neuer Baustein `src/lib/push-endpoint.ts` mit einer Positivliste der bekannten Push-Dienste (FCM, Mozilla, Apple, WNS), `https` erzwungen, keine Anmeldedaten in der URL, kein abweichender Port, keine IP-Literale. Geprüft an **beiden** Enden — beim Speichern und unmittelbar vor dem `fetch`, damit eine Zeile aus Altbestand oder Direktzugriff nicht doch abgerufen wird. Die Migration legt zusätzlich einen `CHECK` auf `https://` in die Datenbank.

CM23-04 (P2) — Push-Nutzdaten gingen unverschlüsselt hinaus, und das Token war unbrauchbar

Zwei Dinge behauptete `src/lib/push.ts`, die es nicht tat:

1. Es setzte `Content-Encoding: aes128gcm` und schickte als Körper **unverschlüsseltes JSON**. Der Zweck der Verschlüsselung in Web Push ist nicht die Leitung — die ist ohnehin TLS —, sondern der **Zustelldienst**: Google, Mozilla und Apple leiten die Nachricht weiter und sollen ihren Inhalt nicht lesen können. Genau das war aufgehoben. Der Inhalt sind Termine, Beträge und Bestellnummern (`Deine Bestellung CM-1042 ist bezahlt (89,00 €)`). 2. Es baute das VAPID-Token mit `createSign('SHA256')`. Das liefert eine **DER**-kodierte ECDSA-Signatur; JWS/ES256 verlangt die rohe Form `r||s` mit festen 64 Byte. Soweit wäre es ohnehin nicht gekommen: der zusammengesetzte PEM-Block bestand aus dem DER-Kopf `30770201010420` und dem 32-Byte-Schlüssel und hörte dann auf. `0x77` = 119 Byte kündigt eine Struktur an, in der neben dem privaten Schlüssel noch die Kurven-OID und der öffentliche Punkt stehen ($3 + 34 + 12 + 70 = 119$). Beides fehlte. `sign.sign()` warf, der Rückfall warf ebenfalls, der Aufrufer zählte `failed++`.

Dazu kommt der Empfänger: **es gibt in der gesamten Anwendung keinen push-Handler**.

`public/sw.js` ist ein Selbstzerstörer, und keine einzige Client-Datei ruft `pushManager.subscribe()` auf. Der Push-Pfad war also an allen vier Stellen gleichzeitig kaputt.

Behoben: neuer Baustein `src/lib/web-push.ts` — RFC 8291 (`aes128gcm` mit ECDH P-256, HKDF, AES-128-GCM) und RFC 8292 (ES256 mit `dsaEncoding: 'ieee-p1363'`, SEC1-DER vollständig gebaut). Der Code sendet lieber gar nichts, als etwas Unverschlüsseltes loszuschicken: ein Konfigurationsfehler bricht die Schleife ab, und die Route antwortet **503** statt `success: true`.

Nachweis: der Test rechnet mit dem Schlüsselmaterail aus **RFC 8291, Abschnitt 5** und belegt zweierlei — der erzeugte Kopf (Salt, `rs`, `idlen`, abgeleiteter öffentlicher Punkt) stimmt Byte für Byte mit dem Beispiel der Norm überein, **und** der private Schlüssel der Norm macht unseren Chiffretext auf und liefert genau ihren Klartext. Für VAPID: die Signatur ist 64 Byte lang und hält einer Prüfung stand; ein nicht zum privaten passender öffentlicher Schlüssel wirft, statt eine Zustellung zu versuchen, die jeder Dienst mit 401 beantworten würde.

CM23-05 (P2) — Die DSGVO-Löschung erreicht die Zustellwege nicht

`notification_log.user_id` und `push_subscriptions.user_id` hängen per `ON DELETE CASCADE` an `profiles`. Nur wird `profiles` in keinem der beiden Löschpfade gelöscht — es wird **anonymisiert** (`email = null`, `full_name = 'Gelöscht'`). Die Kaskade feuert also nie.

Nach einer vollständig durchlaufenen Löschung nach Art. 17 standen damit weiter da:

- das komplette Postfach mit Datum, Betrag und Bestellnummer,
- der Geräte-Endpunkt — ein aktives Zustellziel,
- und die **E-Mail-Adresse in `wait_list`**, die von keinem Löschpfad je angefasst wurde. Die Warteliste kennt kein Konto; sie ist über die Adresse geführt, und eine Abmeldung gibt es dort nirgends.

Behoben: `POST /api/account/delete` löscht beim Antrag die Push-Abos (ein Zustellziel gehört sofort weg, nicht in 30 Tagen) und trägt die Adresse aus der Warteliste aus — dort, wo sie noch bekannt ist, denn dasselbe Update leert `profiles.email`. Der Cron `/api/cron/hard-delete` räumt beim endgültigen Löschen `notification_log` und `push_subscriptions`.

CM23-06 (P2, per Musterabgleich gefunden) — Ein Mietobjekt ließ sich nicht merken

Nach dem ersten 42P10-Befund wurde jede `upsert`-Stelle im Produktivcode gegen ihren Index geprüft (neun insgesamt). Sechs sind in Ordnung (Primärschlüssel oder passender Constraint). Eine war es nicht:

```
.upsert({ customer_id, equipment_id }, { onConflict: 'customer_id,equipment_id' })
```

Der zugehörige Index aus `20260827_favorites_equipment.sql` ist **partiell**:

```
CREATE UNIQUE INDEX uq_favorites_customer_equipment
ON public.favorites(customer_id, equipment_id)
WHERE equipment_id IS NOT NULL;
```

Ein partieller Index kommt als Arbiter nur in Frage, wenn die Anfrage sein Prädikat mitliefert.

PostgREST schickt über `on_conflict=` aber nur die Spaltenliste — also nie. `favorites.equipment_id` ist live vorhanden (Sonde 28.08.2026), die Migration ist mithin eingespielt: **jeder Klick auf „Inserat merken“ lief in 42P10** und kam als 500 „Konnte nicht gemerkt werden“ zurück. Die Salon-Seite derselben Route war nicht betroffen — dort gibt es mit `favorites_customer_salon_unique` einen vollen Constraint.

Behoben: ein reines `INSERT`. Es braucht gar keinen Arbiter, und der Fall „schon gemerkt“ wird in dieser Route seit jeher als Erfolg behandelt (23505).

Nebenbefund: `src/test/live-schema.ts` führte `favorites.equipment_id` nicht und behauptete damit einen Zustand, den die Live-Datenbank seit dem 27.08. nicht mehr hat. Nachgetragen, ebenso in `scripts/schema-probe.sh`.

CM23-07 (P3) — Cookie-Einwilligung: rohe DB-Meldung, kein Deckel, kein Nachweis

`POST /api/cookies/consent` ist absichtlich ohne Anmeldung erreichbar — die Entscheidung fällt, bevor sich jemand anmeldet. Drei Dinge fehlten:

1. `return NextResponse.json({ error: error.message }, { status: 500 })` — die rohe Datenbankmeldung auf einer unangemeldeten Route. Der letzte Nachzügler der Aufräumarbeit aus Track 18/19. 2. Kein eigenes Rate-Limit auf einem unangemeldeten `INSERT`. 3. Die Zeile trug **keinerlei Zuordnung**. `cookie_consent` führt live eine Spalte `ip_hash` (Sonde 28.08.2026) — genau für diesen Zweck, und sie blieb leer. Ein Einwilligungsnachweis, der niemandem zuzuordnen ist, ist als Nachweis wertlos; die `session_id` kommt aus dem Browser und ist frei wählbar.

Behoben: `dbError()` statt roher Meldung, 20/h je IP, und der HMAC der IP wird geschrieben (nicht die IP — dieselbe Linie wie `visit_logs`, `error_logs` und `login_attempts` seit Track 19).

Nachtrag ohne eigene Nummer: Die Kachel „Benachrichtigungen“ im Konto trug als Unterzeile das Wort „**Aktiv**“ — fest verdrahtet, aus keiner Quelle gelesen, und es gibt in der gesamten Oberfläche keinen Schalter, den sie beschreiben könnte. Ersetzt durch eine Beschreibung dessen, was tatsächlich zugestellt wird.

Was ohne Befund blieb

- `/api/notifications` (GET/PUT): Mandantentrennung sauber — beide Zweige filtern auf `user_id` aus der Session, die IDs werden als UUID geprüft, die Menge ist auf 100 gedeckelt, Fehler laufen über `dbError()`.
- **Empfängerwahl aller acht `createNotification` -Aufrufe:** jeder schreibt an die fachlich richtige Seite (Mieter *und* Vermieter bei Storno und Zahlung), und jeder benutzte `type` steht auf der Positivliste des CHECK-Constraints.
- **Rechte auf allen vier Tabellen:** `wait_list`, `push_subscriptions`, `notification_log` und `cookie_consent`s antworten `anon` mit `42501`. Der Riegel steht also schon; die Migration zieht das REVOKE trotzdem explizit nach — dieselbe Linie wie `20260827_anon_grant_lockdown.sql`.
- `/api/push/send`: Admin-Prüfung, UUID- und Längenprüfungen waren in Ordnung (Track 18). Geändert wurde nur die Ehrlichkeit der Antwort.
- `/api/analytics/visit`: seit Track 19/20 sauber, Gegenprobe bestätigt.

Nicht abschließend prüfbar: `customer_salon_history` (`onConflict: 'customer_id,salon_id'`) hat im Repo kein `CREATE TABLE`; ob dort ein passender UNIQUE-Constraint steht, ist ohne DB-Zugang nicht feststellbar. Der aufrufende Code fängt einen Fehler ab und zählt dann hoch, verwechselt dabei aber 42P10 mit „Zeile existiert schon“. Notiert für einen späteren Track.

Migration

`supabase/migrations/20260828170738_benachrichtigungswege_haertung.sql` — **committet, NICHT angewendet**. Diese Session hat keinen Datenbankzugang. Der Ledger (`docs/MIGRATION_LEDGER.md`) ist die Stelle, an der eine Session mit Zugang das Einspielen einträgt.

Inhalt:

1. `push_subscriptions.updated_at` nachtragen. 2. `wait_list`: ein Arbitr-fähiger `UNIQUE NULLS NOT DISTINCT (email, city)` (ab PostgreSQL 15; darunter bleibt es beim Ausdrucks-Index, mit `NOTICE`). 3. Sechs `CHECK`-Constraints für Regeln, die heute **nur** im Route-Handler stehen: Endpunkt `https`, Schlüsselmaterial vorhanden, E-Mail normalisiert und mit `@`, Stadt nie leere Zeichenkette, `choices` mit allen drei Kategorien. 4. `DROP POLICY cookie_consent_insert_anon` — „jeder darf einfügen“ trägt heute nichts, weil `anon` gar kein GRANT hat, und ist genau die Sorte Riegel, die erst auffällt, wenn jemand ein GRANT nachzieht. 5. `REVOKE ALL ... FROM anon` plus `ENABLE ROW LEVEL SECURITY` auf allen vier Tabellen.

Jeder Constraint **zählt vorher die verletzenden Zeilen** und bricht mit klarer Meldung ab, statt still zu ändern. Die Migration löscht nichts und schreibt keine fachlichen Werte. Am Ende steht eine rein lesende Gegenprobe.

Werkzeug

- `src/test/fake-supabase.ts`: `upsert()` nachgetragen — mit der Eigenschaft, an der der Produktivcode gescheitert ist. Fehlt der Arbitr-Index, gibt es **42P10**, keinen stillen Rückfall. Ausdrucks-Indizes sind bewusst gar nicht darstellbar: sie kommen auch in Postgres nicht in Frage.

- `src/__tests__/e2e/_harness/fake-supabase.ts` : `addUniqueIndex()` , `requireArbiterIndex()` (opt-in, weil die Bestandstests ihre Indizes noch nicht registrieren) und echte 23505-Durchsetzung beim `INSERT` — inklusive Postgres' NULL-Semantik und partieller Indizes.
 - `scripts/schema-probe.sh` und `src/test/live-schema.ts` : um `push_subscriptions` , `wait_list` , `cookie_consent`s erweitert; `favorites` korrigiert.
-

Anmerkung zum Ablauf

Während dieser Arbeit hat eine **parallele Session im selben Arbeitsverzeichnis** alle offenen Änderungen mit `git add -A` eingesammelt und als `5af4013` („CM22 Migration PROVEN_LIVE: Ledger aktualisiert“) committet. Inhaltlich ist nichts verloren gegangen, aber neun Dateien dieses Tracks stehen jetzt unter einer Commit-Nachricht, die von etwas anderem handelt. Festgehalten, damit die History lesbar bleibt.

Aus derselben Quelle stammt eine Nachricht, die den Stand ändert: die CM22-Migration ist **live angewendet und verifiziert**. Live nachgeprüft: `rental_equipment` antwortet `anon` inzwischen mit 401 statt mit Daten.